

## DRY, KISS, YAGNI

El mercado tecnológico actual exige una agilidad que las estructuras rígidas del pasado, heredadas de una visión estrictamente académica de la programación orientada a objetos, ya no pueden sostener por sí solas. La eficiencia operativa en el desarrollo de software moderno no reside únicamente en el cumplimiento de patrones de diseño complejos, sino en la capacidad de aplicar un criterio pragmático que priorice la mantenibilidad y la velocidad de entrega. En este contexto, la soberanía técnica del desarrollador senior se manifiesta no mediante la creación de arquitecturas barrocas, sino a través del dominio de principios transversales que desafían la inercia del exceso de ingeniería. Adoptar una mentalidad orientada a la eficiencia implica reconocer que herramientas como Java sentaron las bases de los principios SOLID, pero que los lenguajes contemporáneos ofrecen paradigmas multiparadigma que permiten soluciones mucho más livianas y directas. El verdadero valor empresarial se genera cuando el código deja de ser un monumento al ego del programador para convertirse en un activo dinámico, capaz de evolucionar sin el lastre de abstracciones innecesarias o repeticiones arbitrarias. **Dominar los principios DRY, KISS y YAGNI** permite a los líderes técnicos reducir la deuda técnica antes incluso de que esta se manifieste en el repositorio. Este enfoque prepara para un entorno de incertidumbre donde el requisito de hoy puede ser irrelevante mañana, exigiendo una infraestructura de software que sea tan robusta como flexible. A ello se suma la soberanía técnica de saber cuándo romper las reglas: entender que la duplicidad de código en el dominio de negocio puede ser, en ocasiones, una salvaguarda estratégica contra el acoplamiento destructivo. En última instancia, la excelencia en la ingeniería contemporánea se mide por la simplicidad de sus soluciones y la precisión de sus abstracciones, garantizando que cada línea de código escrita sea estrictamente necesaria para el éxito del producto.

### La Dualidad del Principio DRY: Eficiencia vs. Acoplamiento

El concepto de Don't Repeat Yourself (DRY) se ha interpretado tradicionalmente como un mandato absoluto contra la duplicación. Sin embargo, desde una perspectiva de arquitectura de software avanzada, la repetición no siempre es un fallo; a veces es una decisión de diseño para preservar la independencia de módulos. Cuando dos procesos comparten una lógica idéntica hoy, pero representan entidades de negocio con ciclos de vida diferentes —como un pedido y una factura—, unificarlos bajo una única función crea un acoplamiento peligroso. Si el departamento de logística cambia su forma de calcular impuestos pero el de finanzas no, una abstracción compartida obligará a introducir condicionales complejos, ensuciando el código que originalmente se quería 'limpiar'. El criterio estratégico sugiere aplicar DRY solo cuando la lógica es verdaderamente cohesiva y su cambio será necesariamente simultáneo en todos los puntos de uso.

### Simplicidad como Ventaja Competitiva: El Poder de KISS

#### Reducción de Ruido Cognitivo

El principio Keep It Simple, Stupid (KISS) actúa como un filtro contra la tendencia natural del ingeniero a sobrecomplicar procesos. En un entorno de alto rendimiento, el código que requiere un manual de

instrucciones para ser interpretado es un pasivo financiero. La simplicidad funcional, apoyada por herramientas modernas y el uso de inteligencia artificial para refactorizar pipelines complejos, permite que cualquier miembro del equipo pueda auditar, corregir y mejorar la base de código sin fricciones. La simplicidad no es falta de sofisticación; es el resultado de un proceso de destilación donde solo queda lo esencial para cumplir el objetivo de negocio.

### **Alineación con el Talento del Equipo**

Implementar soluciones tecnológicamente avanzadas pero ajenas a la cultura técnica del equipo — como pasar de un paradigma imperativo a uno funcional puro sin preparación previa— es un error estratégico. El principio KISS también abraza el contexto humano: una solución técnica es 'simple' solo si el equipo que la mantiene puede operarla con confianza. La adopción gradual de nuevas capacidades lingüísticas asegura que la simplificación del código no se convierta en una barrera de entrada para el talento interno.

### **YAGNI y la Gestión de la Sobreingeniería Prematura**

La anticipación mal entendida es una de las mayores fuentes de desperdicio en el desarrollo de productos digitales. El principio You Aren't Gonna Need It (YAGNI) establece que no se debe añadir funcionalidad basada en suposiciones futuras. Introducir campos de datos, métodos o abstracciones 'por si acaso' no solo consume tiempo de desarrollo presente, sino que hipoteca el futuro con validaciones y tests innecesarios. Más crítico aún es el riesgo de crear la abstracción incorrecta: sin datos reales de uso, las estructuras creadas prematuramente suelen ser erróneas, obligando a costosas refactorizaciones cuando la necesidad real finalmente emerge. **La agilidad real emana de la postergación de decisiones** hasta tener la información necesaria.

### **Observaciones Clave**

- Priorizar la legibilidad sobre la brevedad; un código corto no siempre es un código simple.
- Evaluar el impacto del acoplamiento antes de extraer funciones comunes en dominios de negocio distintos.
- Utilizar la inteligencia artificial como un revisor de simplicidad, no solo como un generador de lógica.
- Evitar la 'parálisis por análisis' en la abstracción inicial: es preferible iterar sobre código funcional que diseñar en el vacío.
- Reconocer que el paradigma de programación debe adaptarse al problema y no al revés.

### **Conclusión**

La transición de un desarrollador técnico a un arquitecto estratégico radica en la capacidad de discernir cuándo la ortodoxia debe ceder ante el sentido común. La integración de DRY, KISS y YAGNI no como reglas rígidas, sino como brújulas de decisión, transforma el desarrollo en un proceso de optimización constante de recursos. Al final de la jornada, la calidad del software se mide por su capacidad de generar valor de forma sostenible, minimizando la fricción técnica y maximizando la claridad operativa. Este equilibrio es el que permite escalar sistemas sin que el peso de la propia arquitectura termine por colapsar la capacidad de innovación de la organización.

**BIG** school